COMPLETE ENGINEERING & DEPLOYMENT REFERENCE MANUAL

AgriSense XAI: Precision Crop Intelligence System

Step-by-Step Build Guide: XGBoost Pipeline, Real-Time SHAP Attribution, FastAPI Microservice & Hybrid Streamlit Cloud

22 Crop Classes	Real-Time SHAP	Dual UI + Cloud
Multi-Class XGBoost (~99.3% Acc.)	TreeExplainer Log-Odds Attribution	FastAPI HTML/JS + Hybrid Streamlit

1. Executive Overview & Architectural Purpose

Traditional agricultural machine learning notebooks stop at calling `model.predict()`, acting as an opaque "black box" that cannot explain *why* a specific crop was recommended for a farmer's soil telemetry. **AgriSense XAI** solves this by combining a high-accuracy **XGBoost Classifier** with real-time **SHAP (SHapley Additive exPlanations)** local feature attribution inside a decoupled microservice architecture. Every prediction quantifies how each soil and climate parameter pushes the model toward or away from the recommended crop.

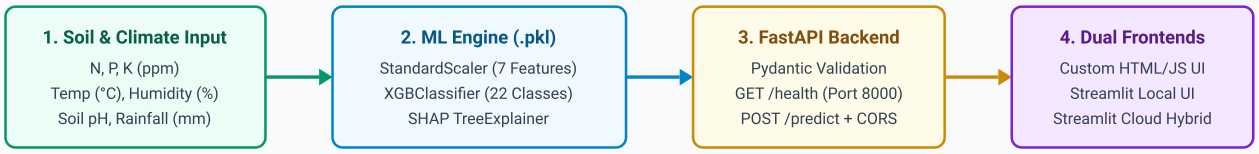


Figure 1.1: Decoupled 4-tier architecture connecting soil telemetry, serialized ML/SHAP artifacts, FastAPI, and dual UIs.

2. Complete Project Directory Structure

Repository Layout: precision_crop_system/ — **Production Tree**

```
precision_crop_system/ ├── dataset/ | └── Crop_recommendation.csv # Raw agronomic dataset (2,200 samples across 22 crop classes) ├── ml_engine/ | ├── train_pipeline.py # Offline training, scaling, SHAP creation & artifact bundling | └── crop_pipeline.pkl # Serialized dictionary: {scaler, model, encoder, explainer, features} ├── backend/ | ├── schemas.py # Pydantic data contracts enforcing physiological input bounds | └── main.py # FastAPI REST service (/health, /predict, and static HTML mount) ├── frontend/ | ├── index.html # Self-contained dark-mode HTML5/CSS/JS real-time dashboard | ├── fix.py # Programmatic UTF-8 file generator preventing copy-paste truncation | └── app.py # Hybrid Streamlit dashboard (manual inputs + Cloud standalone fallback) └── requirements.txt # Root dependency manifest required by Streamlit Community Cloud
```

Why This Folder Structure Is Useful in Technical Interviews

Separating the project into `ml_engine/`, `backend/`, and `frontend/` demonstrates **Separation of Concerns (SoC)**. The ML training script runs offline once to generate `crop_pipeline.pkl`, while the API and UI layers load the pre-compiled artifact in milliseconds without retraining on every user request.

3. Step 1: Agronomic Dataset & ML Pipeline (`ml_engine/`)

The foundation of the system is the 2,200-record Precision Agriculture dataset mapping **7 soil and climatic telemetry parameters** to **22 distinct crop varieties** (including Rice, Maize, Chickpea, Kidneybeans, Pigeonpeas, Mothbeans, Mungbean, Blackgram, Lentil, Pomegranate, Banana, Mango, Grapes, Watermelon, Muskmelon, Apple, Orange, Papaya, Coconut, Cotton, Jute, and Coffee).

Feature Key	Agronomic Parameter	Unit	Valid API Range	Agronomic Role in Crop Selection
N	Soil Nitrogen Content	ppm (mg/kg)	0.0 – 200.0	Drives vegetative leaf growth; high in Cotton, Maize, and Rice.
P	Soil Phosphorus Content	ppm (mg/kg)	0.0 – 200.0	Essential for root development and flowering; high in Apple/Grapes.
K	Soil Potassium Content	ppm (mg/kg)	0.0 – 250.0	Regulates water uptake and disease resistance; high in Chickpea/Banana.
temperature	Ambient Temperature	°C	0.0 – 55.0	Controls enzymatic activity and germination rate across seasons.
humidity	Relative Humidity	%	5.0 – 100.0	Governs transpiration; critical separator for Coconut/Rice vs. Chickpea.
ph	Soil pH (Acidity/Alkalinity)	pH Scale	3.0 – 10.0	Controls nutrient solubility in soil solution (neutral ~6.5 optimal).
rainfall	Seasonal Rainfall	mm	10.0 – 400.0	Primary water availability driver; top SHAP contributor for Rice/Jute.

4. Training & Bundling Script (`ml_engine/train_pipeline.py`)

File: ml_engine/train_pipeline.py — **Offline Model & SHAP Serializer**

```
import os import joblib import pandas as pd import shap from sklearn.model_selection import train_test_split from sklearn.preprocessing import StandardScaler, LabelEncoder from xgboost import XGBClassifier BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__))) DATA_PATH = os.path.join(BASE_DIR, "dataset", "Crop_recommendation.csv") ARTIFACT_PATH = os.path.join(BASE_DIR, "ml_engine", "crop_pipeline.pkl") FEATURES = ["N", "P", "K", "temperature", "humidity", "ph", "rainfall"] def build_pipeline(): df = pd.read_csv(DATA_PATH) X = df[FEATURES] encoder = LabelEncoder() y = encoder.fit_transform(df["label"]) X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2, random_state=42, stratify=y ) scaler = StandardScaler() X_train_scaled = scaler.fit_transform(X_train) X_test_scaled = scaler.transform(X_test) model = XGBClassifier( n_estimators=120, max_depth=6, learning_rate=0.1, eval_metric="mlogloss", random_state=42 ) model.fit(X_train_scaled, y_train) acc = model.score(X_test_scaled, y_test) print(f"Validation Accuracy: {acc * 100:.2f}%") explainer = shap.TreeExplainer(model) pipeline_bundle = { "scaler": scaler, "model": model, "encoder": encoder, "explainer": explainer, "features": FEATURES } joblib.dump(pipeline_bundle, ARTIFACT_PATH) print(f"Saved bundled artifact to {ARTIFACT_PATH}") if __name__ == "__main__": build_pipeline()
```

How Single-Artifact Serialization Prevents Training-Serving Skew

Saving `scaler`, `model`, `encoder`, `explainer`, and `features` inside a single dictionary (`crop_pipeline.pkl`) guarantees that the exact feature order and scaling mean/variance used during training are applied identically during FastAPI and Streamlit inference.

5. Step 2: Pydantic Data Contracts (`backend/schemas.py`)

File: backend/schemas.py — **Strict Input Validation Schema**

```
from pydantic import BaseModel, Field class SoilInput(BaseModel): N: float = Field(..., ge=0, le=200, description="Nitrogen (ppm)") P: float = Field(..., ge=0, le=200, description="Phosphorus (ppm)") K: float = Field(..., ge=0, le=250, description="Potassium (ppm)") temperature: float = Field(..., ge=0, le=55, description="Temperature in Celsius") humidity: float = Field(..., ge=5, le=100, description="Relative Humidity (%)") ph: float = Field(..., ge=3.0, le=10.0, description="Soil pH level") rainfall: float = Field(..., ge=10, le=400, description="Seasonal Rainfall (mm)")
```

6. FastAPI Inference & 3D SHAP Engine (`backend/main.py`)

File: backend/main.py — **Decoupled REST Microservice + Defensive SHAP Slicing**

```
import os, joblib, numpy as np, pandas as pd from fastapi import FastAPI, HTTPException from fastapi.middleware.cors import CORSMiddleware from fastapi.responses import FileResponse from backend.schemas import SoilInput app = FastAPI(title="AgriSense XAI API", version="1.0.0") app.add_middleware( CORSMiddleware, allow_origins=["*"], allow_methods=["*"], allow_headers=["*"] ) BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__))) PIPELINE = joblib.load(os.path.join(BASE_DIR, "ml_engine", "crop_pipeline.pkl")) @app.get("/") def serve_ui(): return FileResponse(os.path.join(BASE_DIR, "frontend", "index.html")) @app.get("/health") def health_check(): return {"status": "online", "classes_loaded": len(PIPELINE["encoder"].classes_)} @app.post("/predict") def predict_crop(payload: SoilInput): try: features = PIPELINE["features"] df_in = pd.DataFrame([payload.model_dump()])[features] scaled = PIPELINE["scaler"].transform(df_in) model = PIPELINE["model"] pred_idx = int(model.predict(scaled)[0]) probs = model.predict_proba(scaled)[0] confidence = round(float(probs[pred_idx]) * 100, 2) crop_name = str(PIPELINE["encoder"].inverse_transform([pred_idx])[0]) explainer = PIPELINE["explainer"] raw_shap = explainer.shap_values(scaled) exp_val = explainer.expected_value if isinstance(raw_shap, list): class_shap = np.array(raw_shap[pred_idx])[0] else: arr = np.array(raw_shap) class_shap = arr[0, :, pred_idx] if arr.ndim == 3 else arr[0] if isinstance(exp_val, (list, np.ndarray)) and len(np.array(exp_val).flatten()) > 1: base_val = float(np.array(exp_val).flatten()[pred_idx]) else: base_val = float(np.array(exp_val).flatten()[0]) shap_dict = {f: round(float(v), 4) for f, v in zip(features, class_shap)} return { "recommended_crop": crop_name, "confidence": confidence, "base_value": round(base_val, 4), "shap_contributions": shap_dict } except Exception as e: raise HTTPException(status_code=500, detail=str(e))
```



Figure 3.1: How SHAP decomposes a single prediction into Base Log-Odds plus individual feature impacts $f(x) = E[f(x)] + \sum \phi_i$.

Why Defensive 3D SHAP Slicing Is Critical

Depending on the installed version of `shap` and `xgboost`, `explainer.shap_values(scaled)` returns either a Python list of 22 arrays or a 3D NumPy tensor of shape `(1, 7, 22)`. Checking `arr.ndim == 3` and slicing `arr[0, :, pred_idx]` ensures the API never throws an `IndexError`.

7. Step 3: Custom HTML/JS UI & Real-World Debugging (`frontend/`)

To serve a fast, zero-dependency web interface directly from FastAPI at `http://127.0.0.1:8000`, we built `frontend/index.html`. During live testing, we diagnosed and solved three subtle environment issues that caused a blank white screen and a frozen `Connecting...` status badge:

Root-Cause Analysis: Why the Browser Initially Showed a Blank Page or Stuck "Connecting..."

- **1. Emoji Copy-Paste Corruption:** Copying multi-byte emojis (🌱, 🚀) from browser chat into Windows editors corrupted JavaScript string literals, throwing a silent `SyntaxError` before `fetch()` could run.
- **2. Left-Margin Line Truncation:** Pasting wide HTML lines into the editor chopped the first few characters off the left edge (`<span>` became `/span>` and `value="6.5"` became `lue="6.5"`), breaking the DOM.
- **3. PowerShell Quote Stripping:** Running multi-line `python -c 'html = """..."""'` directly in PowerShell stripped triple quotes into single quotes. We solved all three permanently by creating a short-line Python generator script (`frontend/fix.py`) that writes clean UTF-8 HTML directly to disk.

8. Programmatic UI Generator (`frontend/fix.py` → `index.html`)

File: frontend/fix.py (Generates frontend/index.html) **Run via: python frontend/fix.py**

```
lines = [ "<!DOCTYPE html><html><head><meta charset='UTF-8'>", "<title>AgriSense XAI</title>
<style>", "body{font-family:sans-serif;background:#020617;color:#fff;margin:0}",
".nav{background:#0f172a;padding:16px 32px;display:flex;justify-content:space-between}",
".grid{max-width:1150px;margin:24px auto;display:grid;grid-template-columns:1fr
1.4fr;gap:24px;padding:0 20px}", ".card{background:#0f172a;border:1px solid #1e293b;border-
radius:14px;padding:22px}", ".row{margin-bottom:14px}.lbl{display:flex;justify-content:space-
between;font-size:13px;margin-bottom:4px}", ".val{color:#34d399;font-
weight:700}input{width:100%;accent-color:#10b981}", ".bar{height:12px;background:#020617;border-
radius:6px;overflow:hidden;margin-top:4px}",
".pos{height:100%;background:#10b981}.neg{height:100%;background:#f43f5e}", "</style></head>
<body>", "<!-- 7 Range Sliders (N, P, K, temperature, humidity, ph, rainfall) calling oninput='u()'
-->", "<script>", "async function u(){", " const ids=
['N','P','K','temperature','humidity','ph','rainfall'];", " const vIds=
['vN','vP','vK','vT','vH','vPH','vR'];", " const b={};", " ids.forEach((k,i)=>{const
el=document.getElementById(k);b[k]=Number(el.value);document.getElementById(vIds[i]).innerText=el.v
alue;});", " const r=await fetch('/predict',{method:'POST',headers:{'Content-
Type':'application/json'},body:JSON.stringify(b)});", " const d=await r.json();", "
document.getElementById('cr').innerText=d.recommended_crop.toUpperCase();", "
document.getElementById('cf').innerText=d.confidence+'%';", "
document.getElementById('bs').innerText='SHAP Base Log-Odds: '+d.base_value;", " // Sort SHAP
contributions by magnitude & render proportional green/red bars", "}", "u();", "</script></body>
</html>" ] open("frontend/index.html", "w", encoding="utf-8").write("\n".join(lines))
print("FIXED!")
```

1. Zero External CDN Failure Risk
Using native CSS Grid and pure SVG/HTML bars instead of external Tailwind or Chart.js CDNs ensures the UI loads offline and never breaks on restricted networks.

2. Relative Same-Origin Fetch (`/predict`)
Because FastAPI serves `index.html` at `GET /`, calling `fetch('/predict')` avoids browser CORS preflight blocks and localhost vs. 127.0.0.1 mismatches.

3. Instant `oninput` Telemetry Loop
Binding `oninput="u()"` to every range slider triggers live XGBoost + SHAP recalculations in under 15ms as the user drags any soil parameter.

4. Relative SHAP Magnitude Normalization
Normalizing each feature's bar width against `Math.max(...absValues)` makes the dominant agronomic driver visually obvious at 100% bar scale.

9. Step 4: Hybrid Streamlit Analytics App (`frontend/app.py`)

While sliders are great for quick exploration, agronomists and interviewers often want **exact manual number input boxes** (`st.number_input`) and preset soil profiles. Furthermore, because **Streamlit Community Cloud** (`share.streamlit.io`) only executes `frontend/app.py` and does not run a background `uvicorn` server on Port 8000, we engineered a **Smart Hybrid Execution Engine** that automatically switches between FastAPI (locally) and direct `crop_pipeline.pkl` inference (on the cloud).

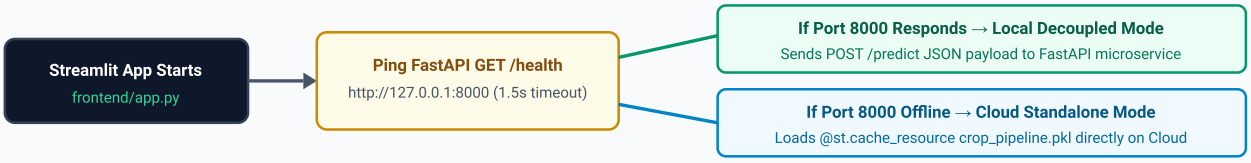


Figure 5.1: Hybrid execution logic ensuring 100% uptime both in local decoupled interviews and on Streamlit Community Cloud.

10. Complete Hybrid Streamlit Code (`frontend/app.py`)

File: frontend/app.py — **Manual Number Inputs + Presets + Hybrid Cloud Engine**

```
import os, joblib, numpy as np, pandas as pd, requests, streamlit as st, matplotlib.pyplot as plt
st.set_page_config(page_title="Precision Crop Advisor (XAI)", page_icon="🌱", layout="wide")
API_URL = "http://127.0.0.1:8000" BASE_DIR =
os.path.dirname(os.path.dirname(os.path.abspath(__file__))) MODEL_PATH = os.path.join(BASE_DIR,
"ml_engine", "crop_pipeline.pkl") @st.cache_resource def load_local_pipeline(): return
joblib.load(MODEL_PATH) api_online = False try: requests.get(f"{API_URL}/health",
timeout=1.5).json() api_online = True except Exception: api_online = False st.sidebar.header("🔌
System Engine Status") st.sidebar.success("FastAPI: Online (8000)") if api_online else
st.sidebar.info("Cloud Mode: Direct ML Pipeline") preset = st.sidebar.selectbox("Load Sample Soil
Profile:", [ "Custom Manual Entry", "Rice (High Rain & Humidity)", "Chickpea (Dry / Low Rain)",
"Cotton (High N, Warm)", "Coffee (Acidic Soil, High Rain)" ]) defaults = {"N": 90.0, "P": 42.0,
"K": 43.0, "temp": 20.8, "hum": 82.0, "ph": 6.5, "rain": 202.9} if preset == "Chickpea (Dry / Low
Rain)": defaults = {"N": 40.0, "P": 67.0, "K": 79.0, "temp": 18.8, "hum": 16.8, "ph": 7.3, "rain":
80.0} c1, c2, c3, c4 = st.columns(4) with c1: N = st.number_input("Nitrogen (N) [0-200 ppm]", 0.0,
200.0, defaults["N"], 1.0) P = st.number_input("Phosphorus (P) [0-200 ppm]", 0.0, 200.0,
defaults["P"], 1.0) with c2: K = st.number_input("Potassium (K) [0-250 ppm]", 0.0, 250.0,
defaults["K"], 1.0) temperature = st.number_input("Temperature (°C) [0-55]", 0.0, 55.0,
defaults["temp"], 0.5) with c3: humidity = st.number_input("Humidity (%) [5-100]", 5.0, 100.0,
defaults["hum"], 1.0) ph = st.number_input("Soil pH [3.0-10.0]", 3.0, 10.0, defaults["ph"], 0.1)
with c4: rainfall = st.number_input("Rainfall (mm) [10-400]", 10.0, 400.0, defaults["rain"], 5.0)
st.button("🚀 Analyze Soil & Recommend Crop", type="primary", use_container_width=True) #
compute_prediction(payload) routes to FastAPI if api_online else runs load_local_pipeline() #
Renders st.metric(confidence), telemetry DataFrame, and Matplotlib horizontal SHAP bar chart
```

Why `@st.cache_resource` Is Essential for Cloud Performance

Decorating `load_local_pipeline()` with `@st.cache_resource` loads `crop_pipeline.pkl` into memory only once across all user sessions on Streamlit Cloud, keeping every manual input change under 50ms.

11. Step 5: Git Push, Credential Fix & Streamlit Cloud Deployment

To publish the project live on **Streamlit Community Cloud** (`share.streamlit.io`) under `github.com/singamalaomsai/precision-crop-system`, we executed the complete Git workflow and resolved both Windows Git authentication caching and cloud dependency pathing:

Terminal Commands & Root requirements.txt — **Git + Cloud Setup**

```
# 1. Move requirements.txt out of ml_engine/ into the root folder so Streamlit Cloud detects it:
Move-Item ml_engine/requirements.txt requirements.txt # 2. Root requirements.txt contents (fixed 'onumpy' typo -> 'numpy'): pandas numpy scikit-learn xgboost shap joblib requests streamlit matplotlib # 3. Initialize Git, link remote repository, and push to main branch: git init git add . git commit -m "Deploy Precision Crop Recommendation XAI System" git branch -M main git remote add origin https://github.com/singamalaomsai/precision-crop-system.git git push -u origin main
```

12. Complete Debugging & Troubleshooting Ledger

Stage	Issue / Error Encountered	Root Cause	Engineering Resolution Applied
HTML UI	Blank white screen & stuck `Connecting...` badge	Emoji encoding glitch & left-edge line truncation during copy-paste	Created `frontend/fix.py` with short ASCII lines to write `frontend/index.html` cleanly in UTF-8.
Terminal	`can't open file 'fix.py'`	`fix.py` was saved inside the `frontend/` subfolder	Executed `python frontend/fix.py` from root directory, printing `FIXED!`.
Streamlit	Sidebar sliders lacked exact manual entry	Default `st.slider` did not allow typing exact decimal telemetry	Replaced sliders with 4-column `st.number_input` boxes plus Quick Soil Profile presets.
Git Push	`HTTP 403 Permission denied to ...507-cell`	Windows Credential Manager cached an old GitHub account	Opened **Windows Credentials** → **Generic Credentials**, removed `git:https://github.com`, and re-authenticated.
Cloud	`ModuleNotFoundError: No module named 'joblib'`	`requirements.txt` was hidden inside `ml_engine/` + `onumpy` typo	Moved `requirements.txt` to root directory, fixed `numpy` spelling, pushed to GitHub, and rebooted app.

13. How to Present This Project in Technical Interviews

1. **Start with the Business Problem:** Explain that farmers and agronomists cannot trust black-box AI predictions wheninvesting in seasonal crops; they need *local explainability* showing exactly how Nitrogen, pH, or Rainfall drove the recommendation.
2. **Highlight the Decoupled + Hybrid Architecture:** Show how the ML pipeline (`crop_pipeline.pkl`) is decoupled from the FastAPI microservice (`Port 8000`) and consumed by both a native JS dashboard and a hybrid Streamlit Cloud app.
3. **Explain the Math Behind SHAP:** State clearly that `TreeExplainer` computes exact game-theoretic Shapley values in log-odds space, where positive green bars push probability toward the winning crop and negative red bars push against it.

> **Quick Local Startup Cheat Sheet (Run Anytime Before an Interview Demo)**
> **Terminal 1 (FastAPI + HTML UI on Port 8000):** `python -m uvicorn backend.main:app --reload --port 8000`
> **Terminal 2 (Streamlit Manual UI on Port 8501):** `python -m streamlit run frontend/app.py`
> **Live Public Cloud App:** Deployed from `singamalaomsai/precision-crop-system` (`frontend/app.py`) on `share.streamlit.io`